

永远相信美好的事情即将发生

xiaomi su7 *Ultra* PROTOTYPE

探秘 Transformer —— 基于 L^AT_EX 刷书法的 笔记整理

作者：Mungeryang

时间：July 15, 2026



目录

1	导言	1
2	注意力机制	2
2.1	概述	2
2.2	背景知识	2
2.3	CNN 和 RNN 方案	4
2.4	注意力机制	5
2.5		5

第 1 章 引言

本篇笔记是基于 $\text{\LaTeX}$ 刷书法，针对博客园大佬“罗西的思考”中系列有关 Transformer 架构剖析的博客内容梳理。

第2章 注意力机制

2.1 概述

最近有些从非 AI 领域转过来的新同学来找我询问是否有比较好的学习资料，他们希望在短期内迅速上手 Transformer。笔者在网上找了下，但是没有找到非常合适的系统的学习资料，于是就萌发了自己写一个系列的想法，遂有此系列。在整理过程中，我也发现了自己很多似是而非的错误理解，因此这个系列也是自己一个整理、学习和提高的过程。

本系列试图从零开始解剖 Transformer，目标是：

- 解析 Transformer 如何运作，以及为何如此运作，让新同学可以入门 Transformer。
- 力争融入一些比较新的或者有特色的论文或者理念，让老鸟也可以通过阅读本系列来了解一些新观点，有所收获。

几点说明：

- 本系列是对论文、博客和代码的学习和解读，借鉴了很多网上朋友的文章，在此表示感谢，并且会在参考中列出。因为本系列参考文章太多，可能有漏给出处的现象。如果原作者发现，还请指出，我在参考文献中进行增补。
- 本系列有些内容是个人的梳理和思考的结果（反推或者猜测），可能和原始论文作者的思路或者与实际历史发展轨迹不尽相同。这么写是因为这样推导让我觉得可以给出直观且合理的解释。如果理解有误，还请各位读者指出。
- 对于某些领域，这里会融入目前一些较新的或者有特色的解释，因为笔者的时间和精力有限，难以阅读大量文献。如果有遗漏的精品文献，也请各位读者指出。

本文为系列第一篇，主要目的是引入 Transformer 概念和其相关背景。在 2017 年，Google Brain 的 Vaswani 等人在论文“Attention is All You Need”中发布了 Transformer。原始论文中给出 Transformer 的定义如下：

Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence aligned RNNs or convolution.

其中提到了 sequence, RNN, convolution, self-attention 等概念，所以我们接下来就从这些概念入手进行分析。我们先开始从 Seq2Seq 介绍，然后逐渐切换到注意力机制，最后再导出 Transformer 模型架构。

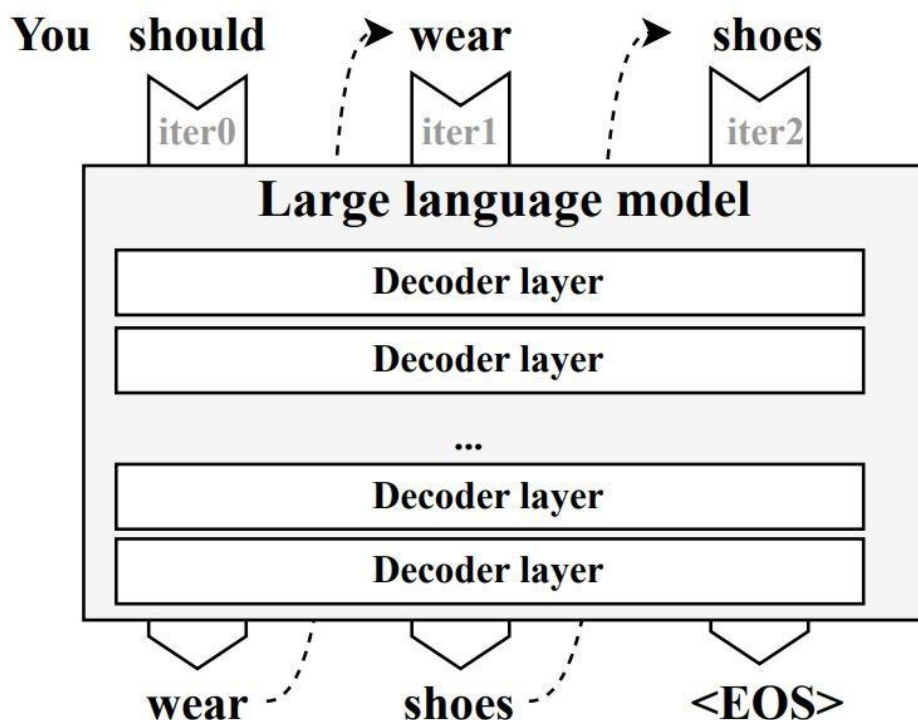
2.2 背景知识

2.2.1 seq2seq

seq2seq (Sequence to Sequence/序列到序列) 概念最早由 Bengio 在 2014 年的论文“Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”中提出，其代表从一个源序列生成一个目标序列的操作。因为机器翻译是大家较熟悉且容易理解的领域，因此后续我们主要使用机器翻译来进行讲解，避免引入过多概念。

2.2.2 文本生成机制

机器翻译其实就是文本生成，语言模型将文本看作是时间序列。在此视角下，每个单词都和它之前的单词相关，通过学习前面单词序列的统计规律就可以预测下一个单词。因此，机器翻译会从概率角度对语言建模，让新预测的单词和之前单词连成整个句子后最合理，即原有句子加上新预测单词后，成为整个句子的概率最大。这就涉及到自回归模型。



图片来源：论文 LLMcad: Fast and Scalable On-device Large Language Model Inference

图 2.1: 自回归模型示例

2.2.3 自回归模型

自回归 (Autoregressive) 模型是一种生成模型，其语言建模目标是根据给定的上下文来预测下一个单词。遵循因果原则（当前单词只受到其前面单词的影响），自回归模型的核心思想是利用一个变量的历史值来预测其未来的值，其将“序列数据的生成”建模为一个逐步预测每个新元素的条件概率的过程。在每个时间步，模型根据之前生成的元素预测当前元素的概率分布。

图 2.1 给出了自回归模型的示例。模型每次推理只会预测输出一个 token，当前轮输出 token 与历史输入 token 拼接，作为下一轮的输入 token，这样逐次生成后面的预测 token，直到新输出一个结束符号或者句子长度达到预设的最大阈值。就图 2.1 来说，模型执行序列如下：

- 第一轮模型的输入是 “You should”。
- 第一轮模型推理输出 “wear”。
- 将预测出的第一个单词 “wear” 结合原输入一起提供给模型，即第二次模型的输入是 “You should wear”。
- 第二次模型推理输出 “shoes”。

该过程中的每一步预测都需要依赖上一步预测的结果，且从第二轮开始，前后两轮的输入只相差一个 token。

自回归模式有几个弊端：

1. 容易累积错误，导致训练效果不佳，因为后面的推理对之前推理的输出会有依赖。在训练初期，模型尚不成熟，几次随机输出会导致随后的训练很难学到任何东西。“一步错，步步错”，训练会变得极不稳定，很难收敛，浪费训练资源。
2. 只能以串行方式进行，这意味着很难以并行化的方式开展训练、提升效率。

2.2.4 编码器-解码器模型

目前，处理序列转换的神经网络模型大多是编码器-解码器 (Encoder-Decoder) 模型。传统的 RNN 架构仅适用于输入和输出等长的任务。然而，大多数情况下，机器翻译的输出和输入都不是等长的，因此，对于输入输出都是变长的序列，研究人员决定使用一个定长的状态机来作为输入和输出之间的桥梁。于是人们使用了一种新

的架构：前半部分的 RNN 只有输入，后半部分的 RNN 只有输出（上一轮的输出会当作下一轮的输入以补充信息），两个部分通过一个隐状态（hidden state）来传递信息。把隐状态看成对输入信息的一种编码的话，前半部分可以叫做编码器（Encoder），后半部分可以叫做解码器（Decoder）。这种架构因而被称为编码器-解码器架构，所用到的模型就是编码器-解码器模型。

2.3 CNN 和 RNN 方案

2.3.1 技术挑战

2.3.1.1 对齐

当面临冗长且信息密集的输入序列时，编码器-解码器模型在整个解码过程中保持相关性的能力可能会减弱。为了更好的说明，我们先看看序列转换面对的主要技术挑战：对齐问题和长依赖问题（或者说是遗忘问题）。

我们来看看为什么要对齐。首先，在某些领域（比如语音识别）中，虽然输入与输出的顺序相同，但是没有一一对应的关系。其次，在某些领域（比如机器翻译）中，在处理完整个输入序列后，模型的输出序列可能和输入序列的顺序不一致。以机器翻译为例，假如我们要让模型将英语“how are you”翻译为中文“你好吗”，或者将“Where are you”翻译成“你在哪里”，我们会发现翻译结果中的语序和原来句子的语序并不相同，同时，一些翻译结果并不能与英语中的词汇一一对应到。

不对齐问题带来的最大困境是：在时间序列的 t 时刻，我们并不能确定此时的模型是否已经获得了输出正确结果所需要的所有信息。因此，人们往往先把所有输入编码到一个隐状态，然后逐步对这个隐状态进行解码，这样才能确保在解码过程中模型一定收到了所需的全部信息。虽然此方案可以保证输入信息的完整性，但却有一个明显缺陷，即在解码过程中，无法确定贡献度。比如，当把“I love you”翻译成“我爱你”时，“我”应该与“I”对齐（因为其贡献最大），但是该方案中，“I”、“love”、“you”这三个词对“我”的贡献都是一致的。

2.3.1.2 长依赖

我们以下面句子为例来进行分析。“秋蝉的衰弱的残声，更是北国的特产，因为北平处处全长着树，屋子又低，所以无论在什么地方，都听得见它们的啼唱。”

将例句从英文翻译成中文时，英文和中文明显是有对齐关系的，因此需要知道哪个英文单词对应到哪个中文。比如将“它们”翻译成“They”；但是“它们”代表什么呢？是“树”？“屋子”？还是“秋蝉”？通过“啼唱”和知识，我们知道“秋蝉”和“它们”指代是同一个对象，但是如果把“听得见它们的啼唱”修改为“看见它们的树荫”，则“它们”指代的就是“树”了。

人类可以很容易的同时看到“秋蝉”和“它们”这两个词然后把这两个词关联起来，即人们知道“它们”和“秋蝉”有长距离的依赖关系，从而理解整个句子。但是对于计算机或者对于模型来说，“秋蝉”和“它们”在例句中的距离太长了，很容易被两个词中间的其它词干扰。为了准确给出最终的答案，神经网络需要对前面“秋蝉的衰弱的残声”和“它们”之间的交互关系进行建模。然而，某些神经网络很难处理长距离依赖关系，因为处理这种依赖关系的关键因素之一是信号在网络中穿越路径的长度，两个位置之间路径越短，神经网络就越容易学习到这种长距离依赖关系，两个位置之间距离越远，建模难度就越大。如果模型无法处理长距离依赖，则会出现长时信息丢失，也就是产生了遗忘问题。

2.3.2 当前问题

我们总结 CNN 和 RNN 这两个方案的主要问题如下：

- 对齐问题。CNN 和 RNN 都难以在源序列和目标序列之间做到完美对齐。
- 隐状态长度固定。这个问题点主要存在于 RNN，因为其隐向量大小固定，所以推理效果受限于信息压缩的能力，导致信息遗失。

- 关系距离问题。此问题在 RNN 和 CNN 中都存在。序列中两个词之间的关系距离不同，当词之间距离过长时，两个方案都难以确定词之间的依赖关系。使得当面临冗长且信息密集的输入序列时，模型在整个解码过程中保持相关性的能力可能会减弱。

2.4 注意力机制

注意力（Attention）机制由 Bengio 团队 2015 年在论文“NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE”中提出，其主要思路为通过模仿人类观察事物的行为来降低算法复杂度、提高性能。人类在感知、认知和行为决策过程中会选择性地关注和处理相关信息，从而提高认知效率和精度，比如人类可以依据兴趣和需求选择关注某些信息而忽略或抑制其它信息，并且可以在多任务上分配注意力从而达到同时处理多个信息的目的。大家最熟悉的例子就是，当一个人看图片时，他会先快速通览，然后对重点区域进行特殊关注。而在文本生成的每个阶段，并非输入上下文的所有片段都同样重要。比如在机器翻译中，句子“A boy is eating the banana”中的“boy”一词并不需要了解整个句子的上下文之后，再进行准确翻译。

注意力机制的本质可以用一句话来总结：上下文决定一切（context is everything）。一个字 / 词在一篇文本中表达的意思通常与它的上下文有关。比如下面两个句子中都有“transformer”这个单词，但是第一个“transformer”应该翻译成“变压器”，第二个“Transformer”最好不做翻译。

2.5